Dynamic Firewall

Dheemanth J PES2UG25AM080 Kishan PES2UG25AM131 Kevin PES2UG25AM128

https://github.com/dj3107/Dynamic-Firewall.git

UE25CS243A · Computer Networks mini project · Project #1 · Mininet · Ryu · OpenFlow 1.3 · Python TCP sockets

Abstract

Conventional firewalls are **static**: their rules are written by hand, in advance. This project builds a **dynamic firewall** on a Software-Defined Network emulated in Mininet, with Open vSwitch as the switch and Ryu as the OpenFlow 1.3 controller. A Python TCP log server on h1 classifies every client connection against a JSON policy. When a client floods the logging service (more than 8 messages in 5 seconds), opens the allow-listed admin port without permission, or exceeds 3 Mbit/s as measured by the switch, the server asks Ryu, through its REST API, to install a drop rule, so the offending traffic is discarded at the switch before it reaches the server. Rules lift themselves once the host goes quiet, trusted hosts are exempt, and a reconcile loop restores rules after a controller restart. Five test scenarios (A to E) and iperf/ping experiments show that the firewall installs a block about 3 ms after deciding, cuts attack traffic by 89% to 97%, never blocks the legitimate host, and recovers with no manual intervention.

Objectives

1. **Detect** misbehaving clients both in the application (the TCP socket server counts messages per host) and in the network (the switch's own flow statistics).
2. **Enforce in the data plane:** install OpenFlow drop rules through Ryu's REST API so attack traffic is dropped at the switch, not at the server.
3. **Respond proportionately:** block a flooding host completely, but deny only the single forbidden flow when a host knocks on the admin port.
4. **Protect what must not be blocked:** exempt trusted hosts with a higher-priority allow rule, and never block the server or the controller.
5. **Recover automatically:** remove blocks after a quiet period and re-install rules after a controller crash, with no human involved.
6. **Measure effectiveness:** the legitimate user's throughput and latency under attack, reaction time, precision and overhead.

What's inside

	Chapter	What it covers
1	At a glance	the idea and the headline results
2	How it works	the parts, what the dashboard shows, and the life of one block
3	The rules	the four rules, one flowchart, priorities and settings
4–8	Scenarios A to E	each one: what happens, the terminal, and the browser before / during / after
9	How effective is it?	speed, protection, precision, resilience and cost, with measurements
10	Limits and verdict	what it does not do, a known issue, and the bottom line

Test setup: the project runs on an Ubuntu VM in Oracle VirtualBox on a Mac. VirtualBox's NAT forwards port 2222 on the Mac to the VM's ssh port (22). Every command was run from the macOS Terminal over ssh, with a tunnel (-L) that carries the dashboard on port 8000 to the Mac's browser:

```
ssh -p 2222 -L 8000:localhost:8000 pes2ug25am080@127.0.0.1
```

With the tunnel open, the dashboard is viewed at **http://localhost:8000** in the Mac's browser.

CHAPTER 1

At a glance

A firewall that reacts while the network is running

A normal firewall is **static**: its rules are written in advance, by hand. This project is a firewall that **changes itself while the network runs**. A small server on h1 watches its own clients. When one misbehaves, the server asks a **controller** (Ryu) to change the **switch's** rules, so the bad traffic is dropped at the switch before it reaches the server. When the client calms down, the rule removes itself.

Detect

The log server counts messages per client and checks each port's allow-list. In scenario E the switch's own counters detect.

Decide

The server tells Ryu what to do over a REST API: block this host, deny this flow.

Enforce

Ryu installs an OpenFlow rule. The switch drops the traffic at once.

Recover

After a quiet period the rule is removed. Nobody has to.

The headline results

1–4 ms

alert → drop rule

from the server noticing to the switch dropping

−97%

attack traffic

reaching the server under the heaviest flood (0.40 → 0.01 Mbit/s)

99.7%

of h2's speed kept

under the same flood, vs 96.2% with no response

0

false blocks of h2

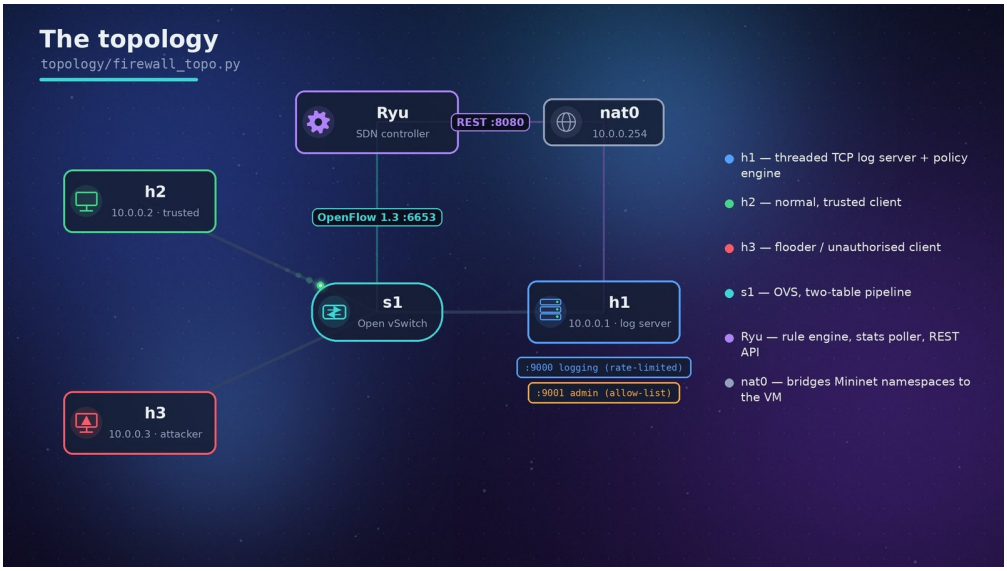
the trusted host was never blocked in any run

How to read these: the firewall clearly works and reacts in milliseconds, but the floods used in the experiments were light, so the benefit to the legitimate user is real yet small (a few percent). Chapter 9 explains this honestly.

CHAPTER 2

How it works

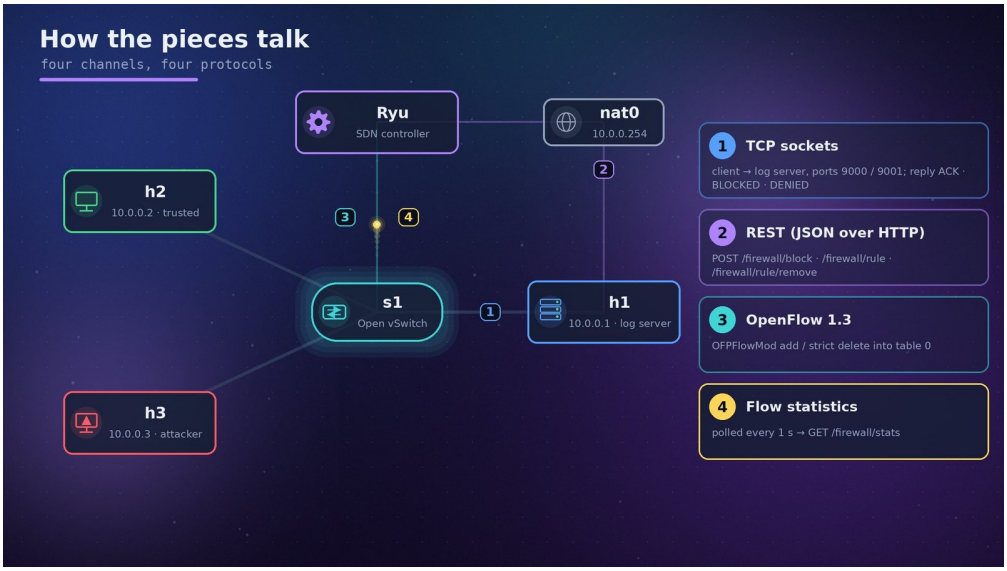
Three hosts, one switch, one controller, four conversations



The emulated network (Mininet).

Piece	Role
h1 · 10.0.0.1	the **log server**: a threaded TCP server that listens on port 9000 (logging) and 9001 (admin). It classifies every client and is also the policy engine.
h2 · 10.0.0.2	a well-behaved, **trusted** client.
h3 · 10.0.0.3	the **troublemaker**: floods the server or knocks on the admin port.
s1	**Open vSwitch**: carries every packet and holds the rules. Speaks OpenFlow 1.3.
Ryu	the **SDN controller**: the only thing allowed to change the switch's rules. It learns MAC addresses, polls statistics every second and offers a REST API on port 8080.
nat0 · 10.0.0.254	a door from the isolated Mininet hosts to the VM, so h1 can reach Ryu's REST API.

Four conversations



Four channels, four protocols.

1	2	3	4
TCP sockets	**REST (HTTP + JSON)**	**OpenFlow 1.3**	**Flow statistics**
clients → log server; replies ACK, BLOCKED or DENIED	log server → Ryu: what to do	Ryu → switch: how to do it	switch → Ryu → server, every second

Inside the switch: two tables

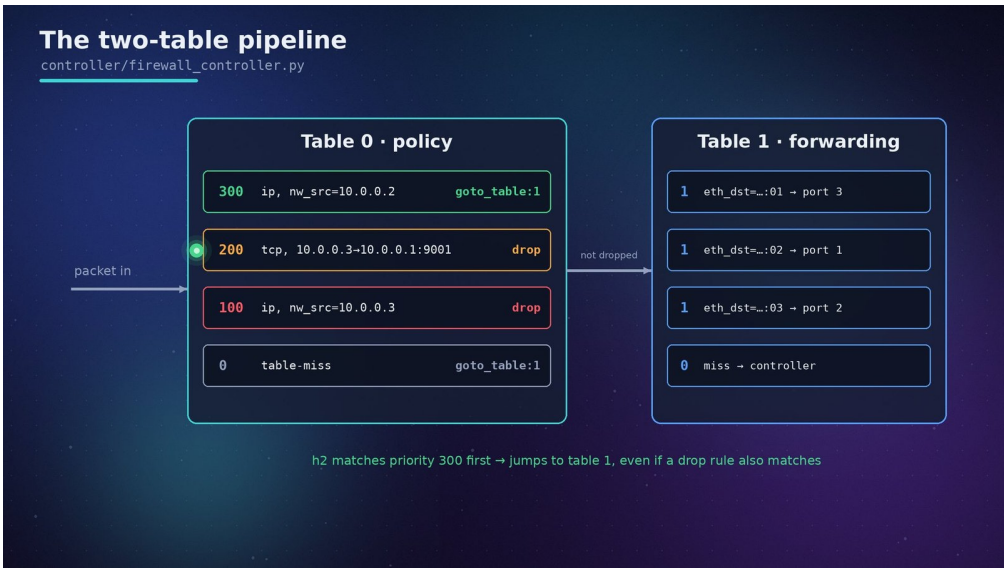
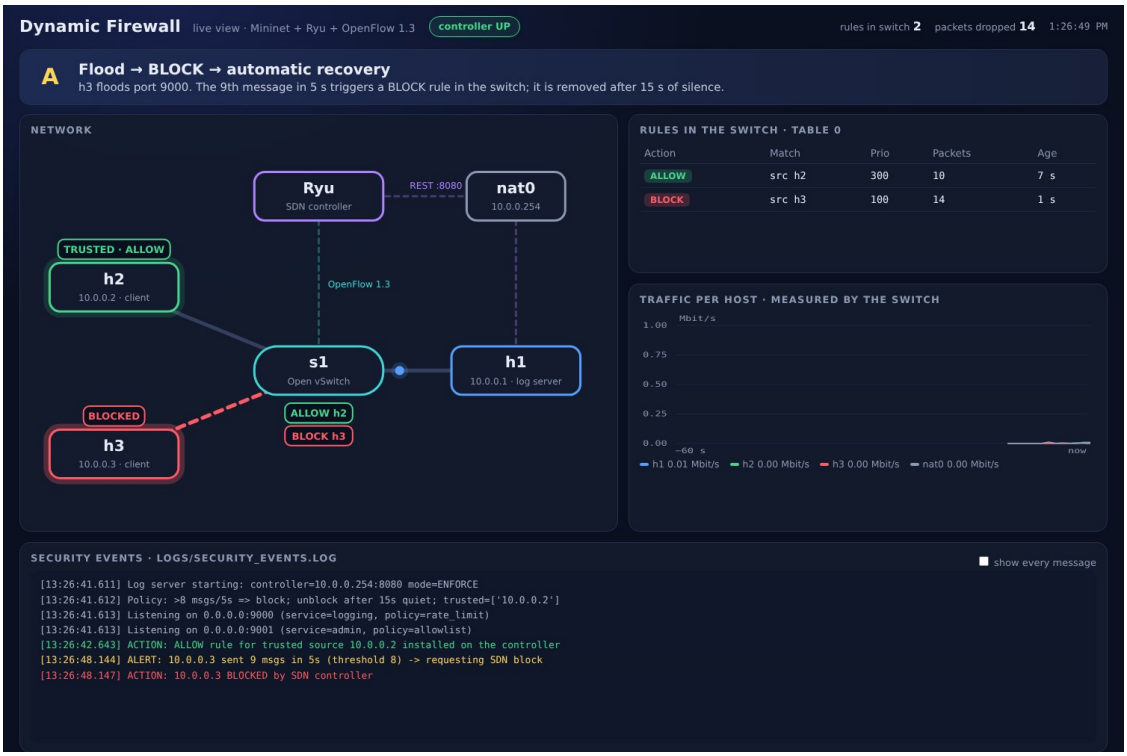


Table 0 holds the firewall rules; whatever is not dropped goes on to table 1.

Table 0 (policy) holds the allow, deny and block rules; the highest priority that matches wins. Anything not dropped continues to **table 1 (forwarding)**, where the controller's learned MAC-address entries live. An ALLOW rule simply says “skip to table 1”, so a trusted host bypasses the drop rules but is still forwarded normally.

What the dashboard shows



The live dashboard (in the Mac's browser, through the ssh tunnel) during a block.

Panel	What it shows
Banner (top)	which scenario is running and what to watch
Network (left)	the topology: a **blocked** host turns red with a dashed link and a BLOCKED badge; a **trusted** host has a green TRUSTED badge; a **denied** flow is marked on its link; packets move along the links as traffic flows; the controller turns red if it is down
Rules in the switch (top right)	every rule currently in table 0 with its priority, live packet counter and age. New rules flash.
Traffic per host (right)	the throughput of each host **measured by the switch**, over the last 60 seconds
Security events (bottom)	the log server's alerts and actions as they happen

Behind the scenes

- **Per-host counting:** the log server keeps a 5-second sliding window of message times for every non-trusted host.
- **Every second:** it expires finished blocks and denies, and (in scenario E) reads per-host throughput from Ryu.
- **Every 2 seconds:** a *reconcile* loop compares Ryu's rule list with what the server expects and re-installs anything missing.
- **Circuit breaker:** if Ryu is unreachable, the server logs **one** warning, keeps serving clients without stalling, and retries every 5 seconds.
- **On a switch (re)connect,** Ryu clears its learned forwarding entries (table 1 only, never the firewall rules), so it relearns which MAC belongs to which IP and per-host statistics keep working.

CHAPTER 3

The rules

Four triggers, three kinds of rule

Every client connection is classified by **source IP, destination IP and port** against `config/policy.json`. The server listens on two TCP ports: **9000** (the logging service) and **9001** (a private admin service).

BLOCK

priority 100

Drops **everything** from one host.

Trigger: more than 8 messages in 5 s on port 9000, **or** (scenario E) the switch measures the host above 3 Mbit/s.

Lifted after 15 s of silence.

DENY

priority 200

Drops **one flow**: source, destination, TCP, port.

Trigger: a host not on the allow-list uses admin port 9001.

Lifted after 30 s.

ALLOW

priority 300

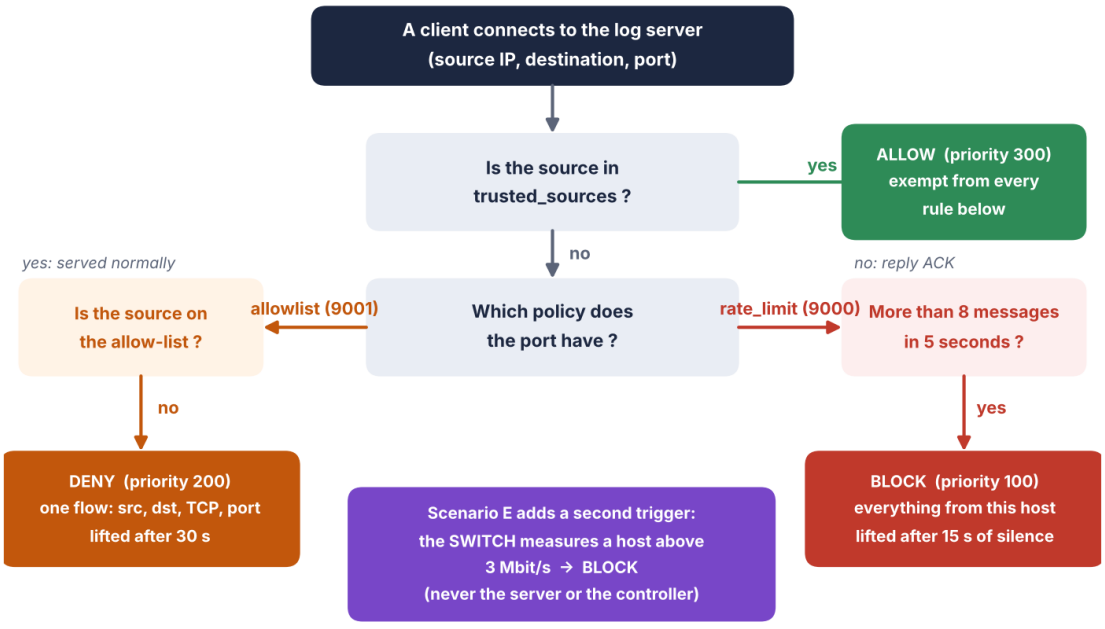
Skips the drop rules and rate limiting.

Trigger: the source is in `trusted_sources` (h2).

Stays while the server runs.

Higher priority wins: allow (300) beats deny (200), deny beats block (100), block beats normal forwarding (1). So even a deny rule on h2 would be ignored: its allow rule matches first.

How one connection is decided



Classification of a connection.

The four rules in detail

Rule	Applies to	Trigger	Effect in the switch	Ends
1 · Trusted	any port	source in `trusted_sources`	ALLOW, prio 300: `ip,nw_src=10.0.0.2 → goto_table:1`	stays while the server runs
2 · Rate limit	port 9000, non-trusted	> 8 messages in 5 s (sliding window)	BLOCK, prio 100: `ip,nw_src=10.0.0.3 → drop`	15 s after the host's last message
3 · Allow-list	port 9001, non-listed	any connection	DENY, prio 200: `tcp,nw_src=…,nw_dst=…,tp_dst=9001 → drop`	after 30 s
4 · Bandwidth	all non-trusted hosts (scenario E)	switch measures ≥ 3 Mbit/s	BLOCK, same as rule 2	15 s of silence

Detail	What happens
The server and controller are protected	Rule 4 never blocks h1 (the server) or the controller's address. The server's own replies to a flood also count as traffic; blocking it would cut it off from Ryu so the rules could never be removed.
Rate limiting only on port 9000	Port 9001 uses the allow-list instead, so a flood on 9001 is DENIED, not rate-limited.
A trusted host is exempt from rules 2, 3 and 4	Allow is an operator decision made in policy.json, not by traffic.
Block is the heavy response, deny the precise one	A blocked host loses all connectivity (even ping); a denied host loses only that one port.
TCP only for DENY	DENY matches TCP; BLOCK matches all IP traffic from the host, which is why a blocked host cannot ping either.

The policy file

```
{ "listen_ports": [9000, 9001],
  "rate": {"window_seconds": 5, "threshold": 8, "recovery_seconds": 15},
  "trusted_sources": ["10.0.0.2"],
  "services": {
    "9000": {"name": "logging", "policy": "rate_limit"},
    "9001": {"name": "admin", "policy": "allowlist",
             "allowed_sources": ["10.0.0.2"], "deny_seconds": 30}}}
```

Changing a number here changes the rule for every scenario. The bandwidth limit (rule 4) is a start-up option of the log server: `--bw-threshold-mbps 3`.

Ports used by the project (all TCP)

Port	What it is
9000, 9001	the log server's two services on h1
6653	OpenFlow: switch ↔ Ryu
8080	Ryu's REST API
8000	the dashboard
22 / 2222	ssh on the VM, and the port on the Mac that forwards to it

SCENARIO A

Flood → BLOCK → recovery

Rule 2: too many messages

What it tests	A client that sends faster than the policy allows is cut off, then forgiven.
What happens	h3 sends a message every 0.1 s to port 9000. The first 8 get `ACK`. The 9th within 5 seconds crosses the limit.
Rule used	Rule 2 → BLOCK (priority 100) on h3. Lifted after 15 s of silence.

In the terminal

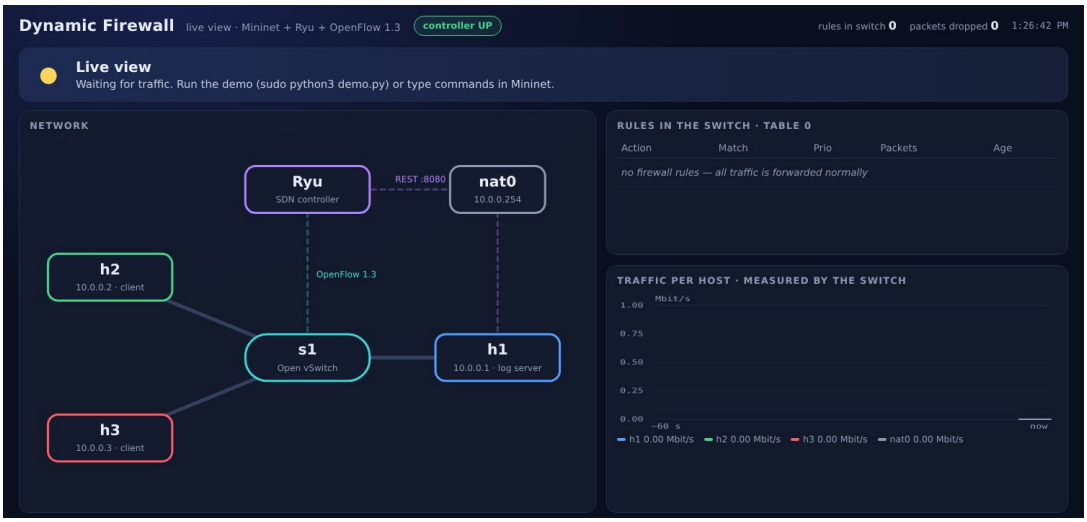


Eight ACKs, then BLOCKED and a timeout; the alert and the drop rule; the failed ping while blocked; then recovery.

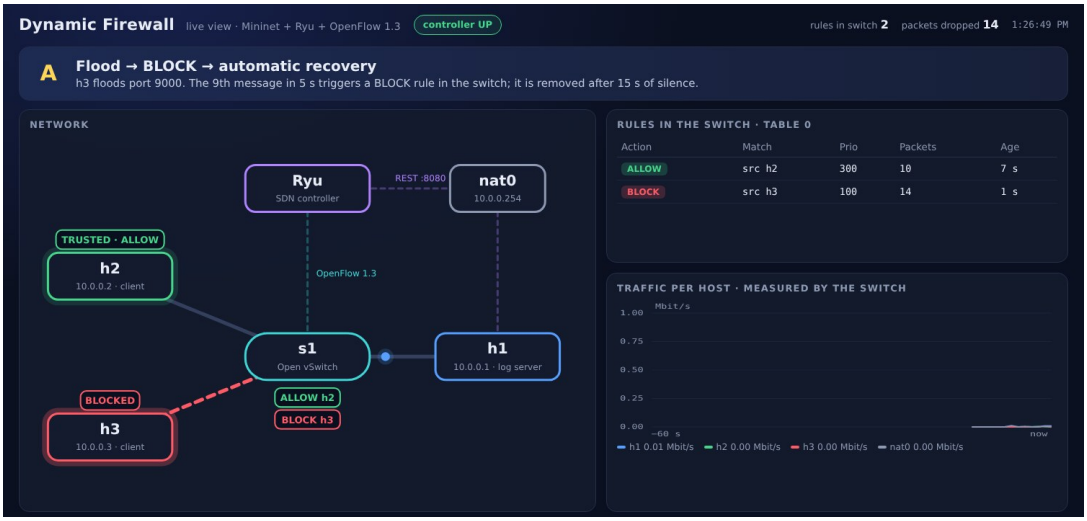
The log, step by step

Time	What happened
13:26:48.144	**ALERT:** 10.0.0.3 sent 9 messages in 5 s (threshold 8) → requesting an SDN block
13:26:48.147	**BLOCKED** by the controller, **3 ms later**: the rule is in the switch
13:27:03.691	**UNBLOCKED** after 15.5 s of silence

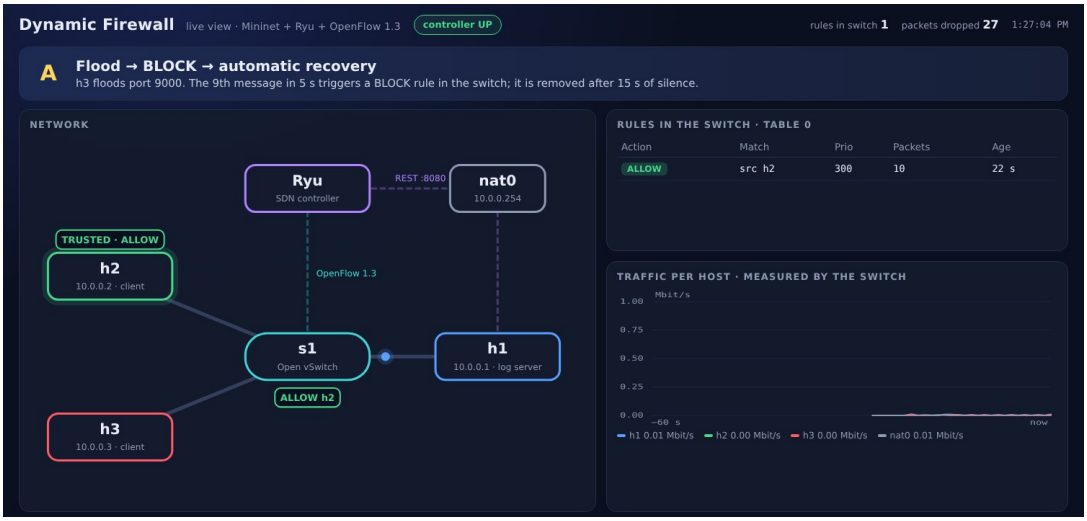
In the browser



Before: only h2's ALLOW rule. Nothing is blocked.



During: h3 is red with a dashed link and a BLOCKED badge. A BLOCK row has appeared, and its packet counter is climbing as the switch discards h3's packets.



After: h3 went quiet; 15 seconds later the BLOCK row and the red marking are gone. Nobody removed them by hand.

Result: the attack was stopped in milliseconds, at the switch, with no human involved, and the network recovered on its own. A blocked host cannot even ping the server.

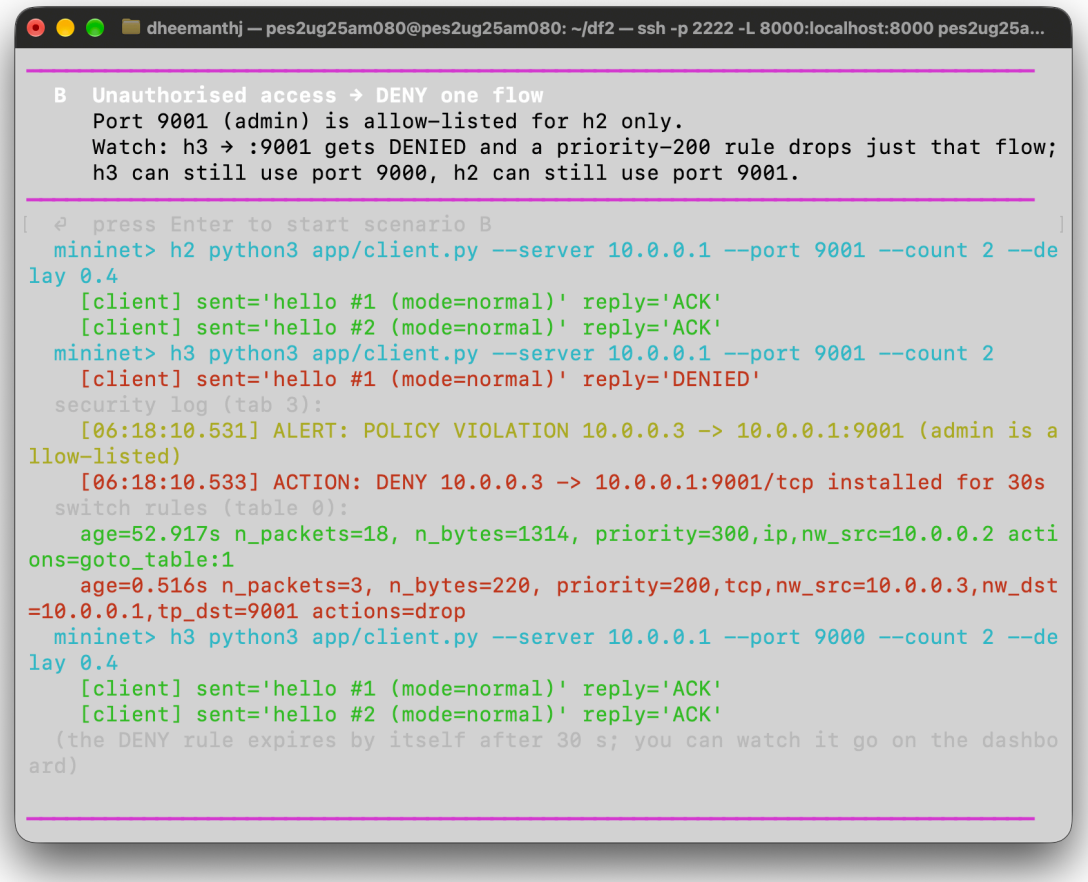
SCENARIO B

Unauthorised access → DENY one flow

Rule 3: the wrong door

What it tests	A client using something it is not allowed to use loses only that, not everything.
What happens	Port 9001 is the private admin service, for h2 only. h2 uses it fine. h3 tries it and gets `DENIED`. Then h3 uses port 9000, which still works.
Rule used	Rule 3 → DENY (priority 200): h3 → h1, TCP, port 9001. Lifted after 30 s.

In the terminal

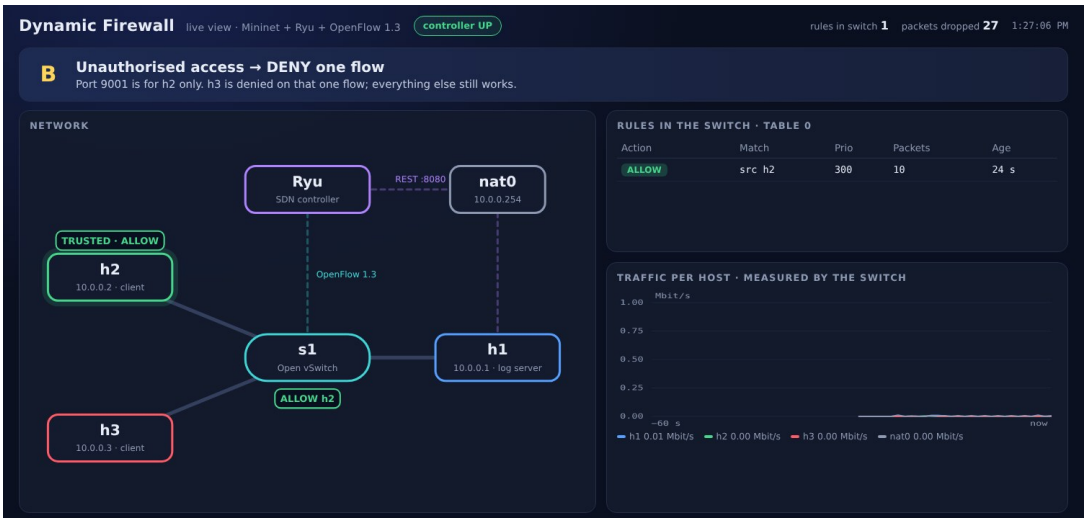


h2 gets ACK on 9001; h3 gets DENIED; the policy-violation alert and the DENY rule; h3 still gets ACK on port 9000.

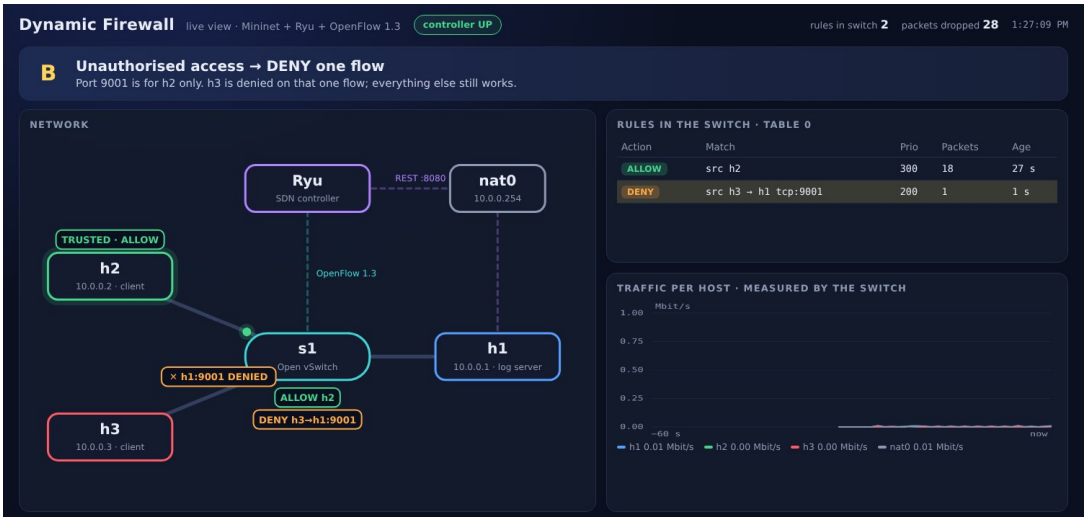
The log, step by step

Time	What happened
13:27:08.676	**ALERT:** policy violation 10.0.0.3 → 10.0.0.1:9001 (admin is allow-listed)
13:27:08.679	**DENY** installed for 30 s, **3 ms later**

In the browser



Before: only the ALLOW rule.



*After: a DENY row (priority 200) for h3 → h1 tcp:9001 has appeared (highlighted as new) and an orange "DENIED" badge marks that one flow on h3's link. **h3 itself is not red:** it is not blocked, only one door is shut.*

Result: the response is proportionate. Block is the heavy response to a misbehaving host; deny is a surgical one. Only the forbidden flow was dropped, everything else h3 does still works.

SCENARIO C

Trusted host → ALLOW

Rule 1: the exemption

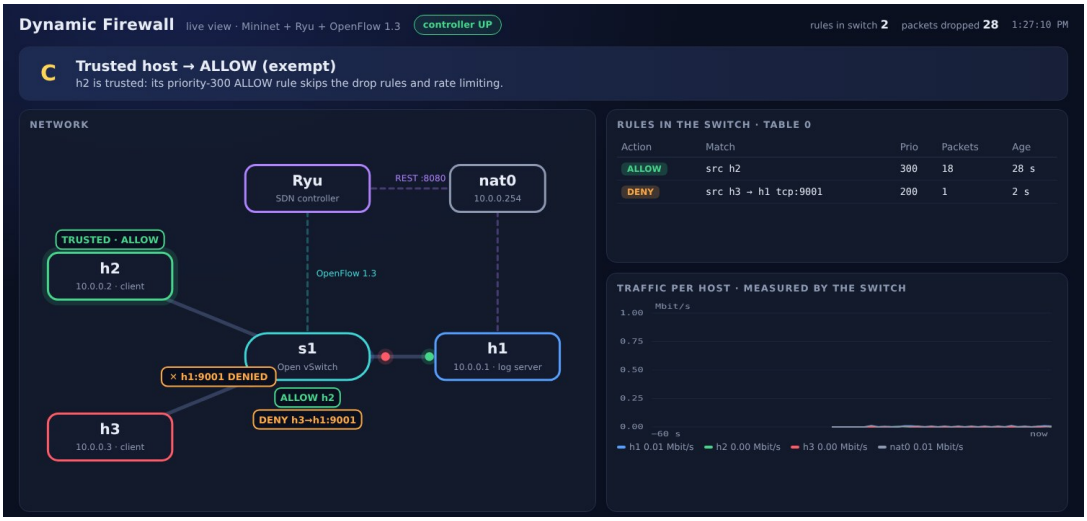
What it tests	A trusted host is never punished, even for behaviour that would get anyone else blocked.
What happens	h2 sends the very same 40-message flood that got h3 blocked in scenario A.
Rule used	Rule 1 → ALLOW (priority 300): `ip,nw_src=10.0.0.2 → goto_table:1`. It matches first and skips the drop rules and the rate limit.

In the terminal

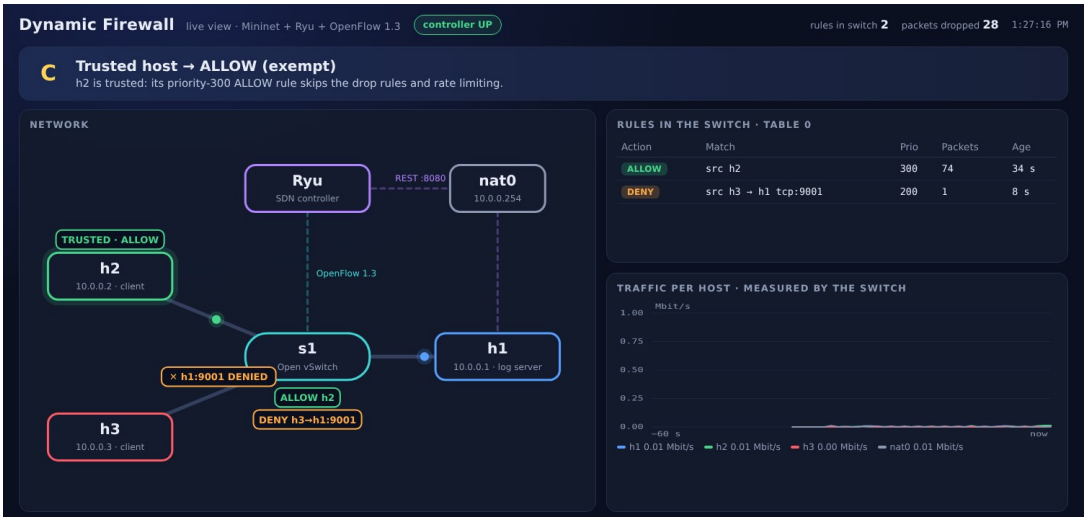


Every reply is ACK: 40 out of 40.

In the browser



Before the flood: the ALLOW rule for h2 has counted 18 packets.



*After: the same ALLOW row has counted **74** packets: h2's flood passed straight through it. Nothing is blocked and h2 is still marked TRUSTED.*

Result: an allow rule overrides the drop rules and the rate limit. This is by design: allow is an operator decision made in `policy.json`, not something traffic can earn.

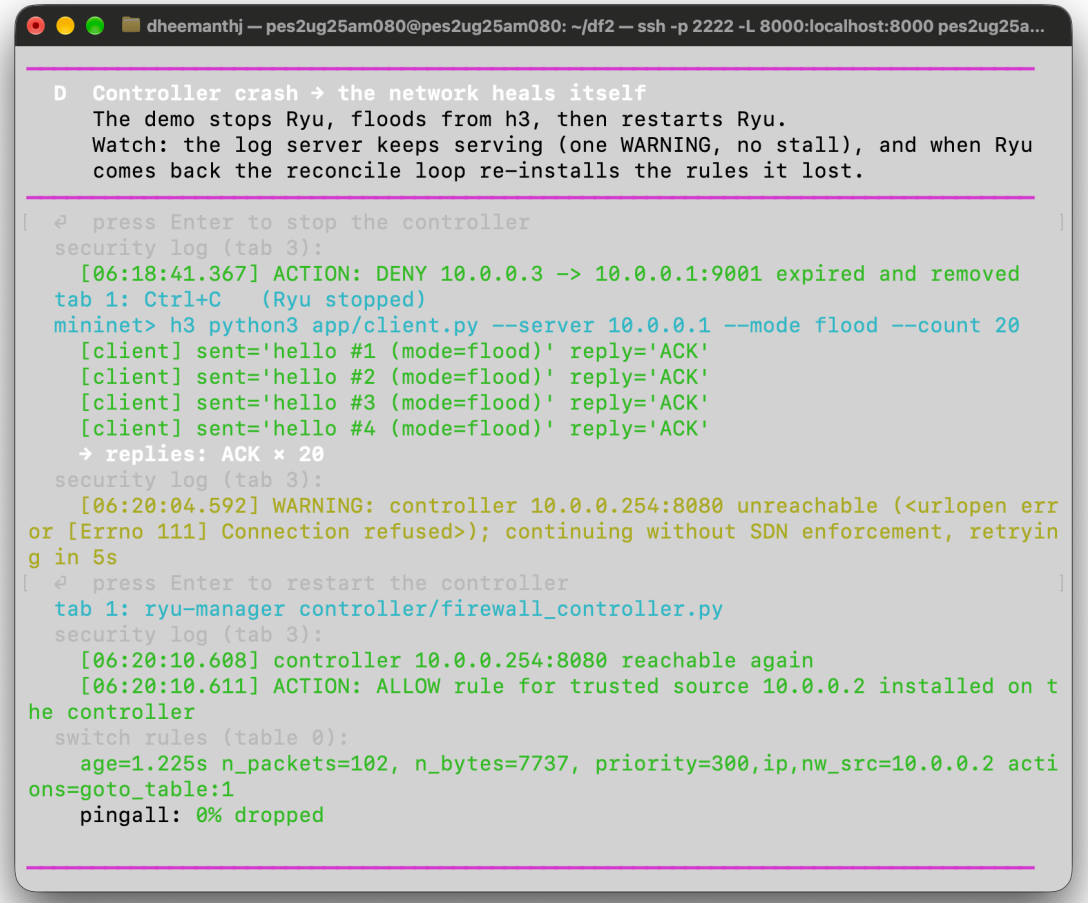
SCENARIO D

Controller crash → the network heals itself

Not a rule: resilience

What it tests	What happens to the firewall if the controller dies, and whether it recovers without help.
What happens	The demo stops Ryu. h3 floods, but nothing can be blocked now. Then Ryu restarts with an empty rule table.
Mechanism	Circuit breaker (one warning, no stall, retry every 5 s) and the 2-second reconcile loop that re-installs missing rules.

In the terminal

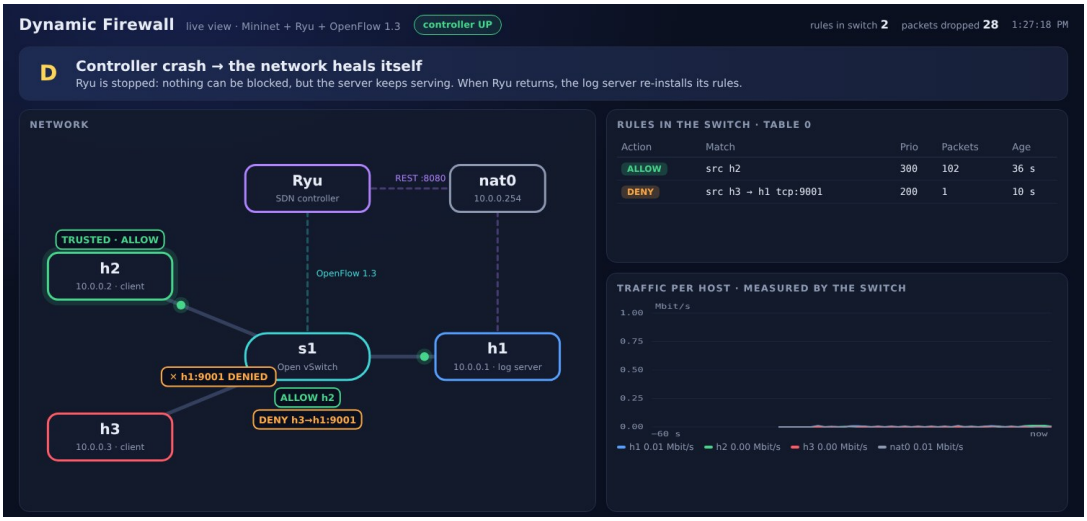


With Ryu down, every reply stays ACK and the log shows one warning. After the restart the rules are re-installed and pingall reports 0% dropped.

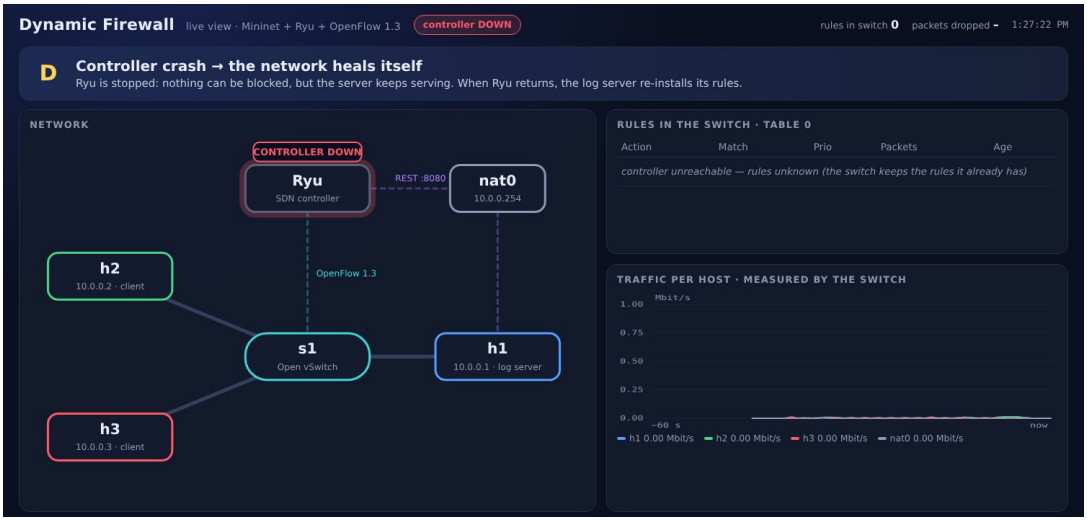
The log, step by step

Time	What happened
13:27:20.738	**WARNING:** controller unreachable; continuing without SDN enforcement (logged once)
13:27:26.744	controller reachable again
13:27:26.747 – .748	**ALLOW** and **DENY** rules re-installed: **3–4 ms** after the controller returned

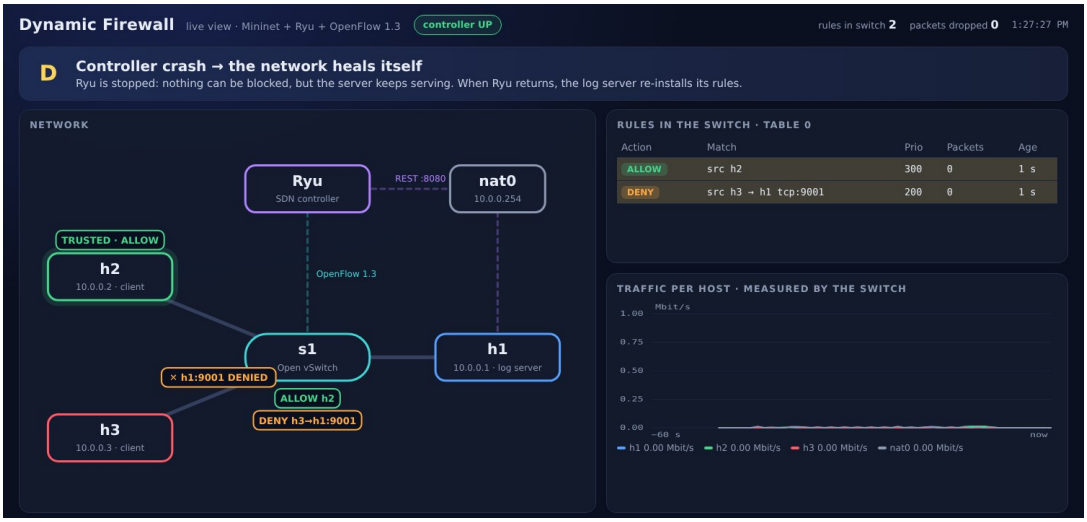
In the browser



Before: controller up, ALLOW and DENY rules in the switch.



Ryu stopped: "controller DOWN" in red; the rules table says the rules are unknown (the switch keeps the rules it already has).



Ryu is back: "controller UP", and both rules are in the switch again with fresh counters. The server put them back.

Result: the service never stalled, nothing was lost silently, and the rules came back by themselves. While the controller is down no *new* blocks can happen, but rules already in the switch keep working.

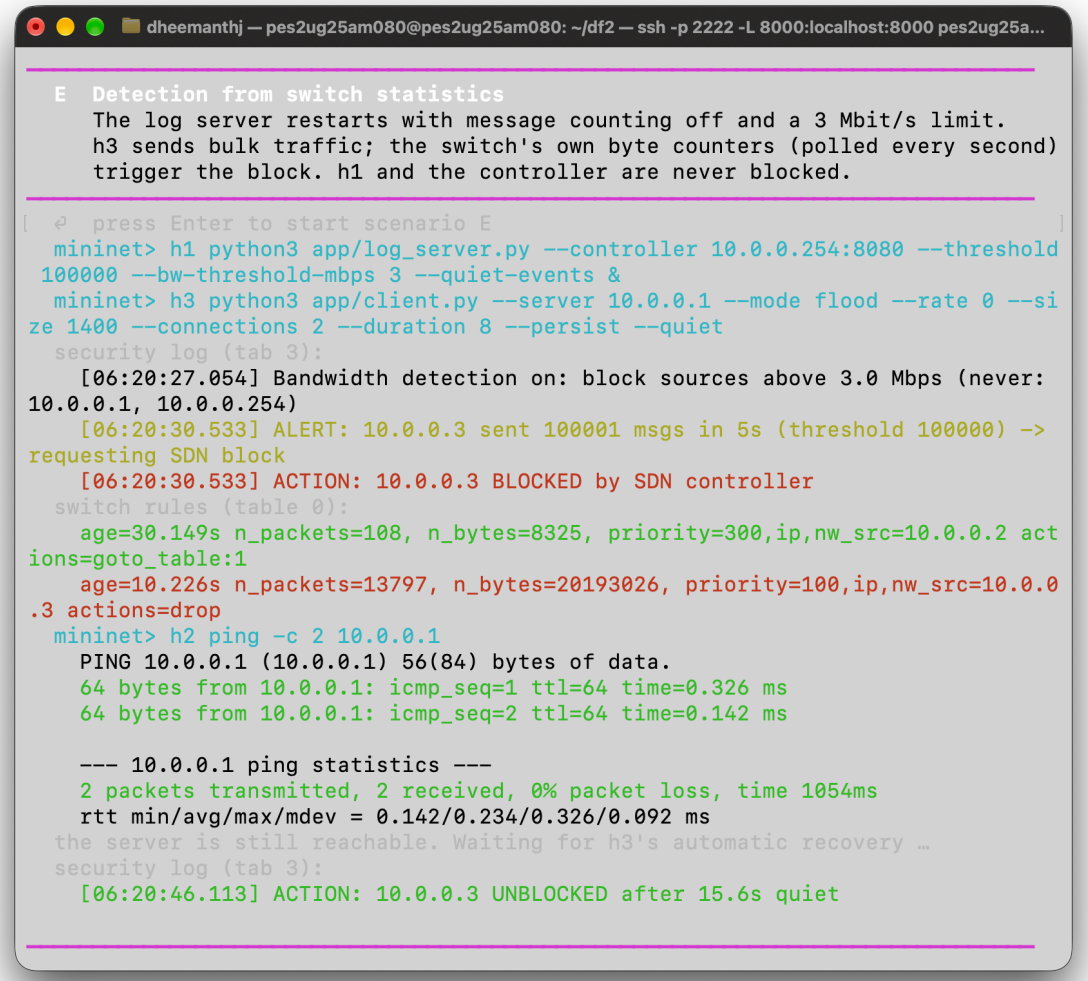
SCENARIO E

Detection from switch statistics

Rule 4: the switch does the measuring

What it tests	That a block can be triggered by what the **network** measures, not only by what the application counts.
What happens	The log server restarts with message counting effectively off (limit 100000) and a bandwidth limit of 3 Mbit/s. h3 sends large messages as fast as it can. Ryu reads the switch's byte counters every second.
Rule used	Rule 4 → BLOCK, same as rule 2. The server and controller are never blocked, so the network cannot lock itself up.

In the terminal

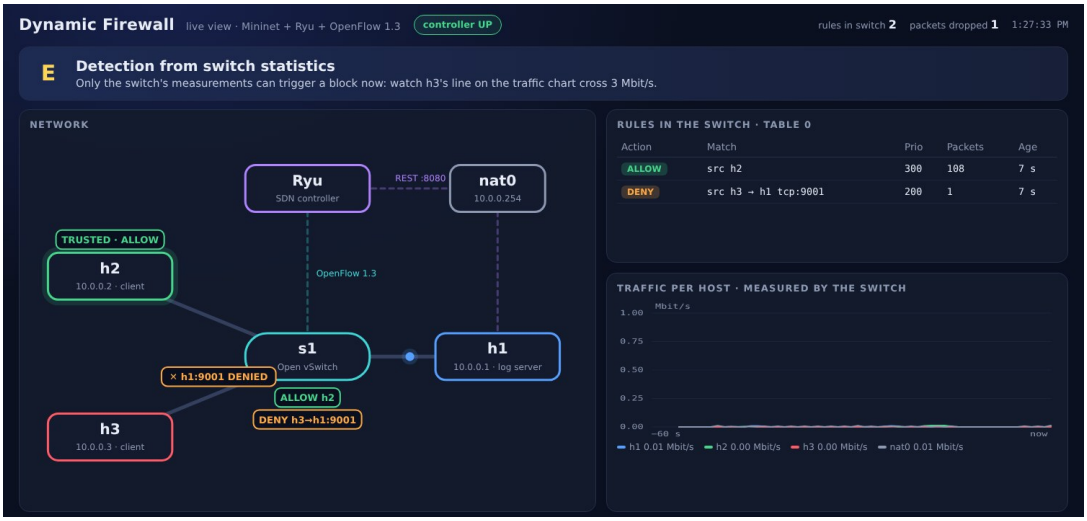


"Bandwidth detection on … (never: 10.0.0.1, 10.0.0.254)", then the alert, the block, a successful ping from h2, and the unblock 15 s later.

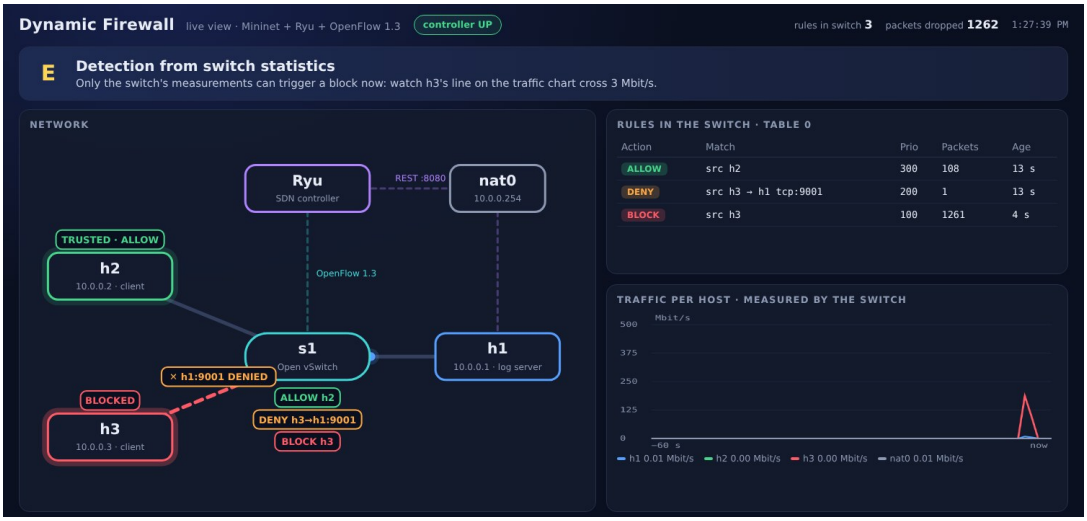
The log, step by step

Time	What happened
13:27:31.733	log server restarts: "Bandwidth detection on: block sources above 3.0 Mbps (never: 10.0.0.1, 10.0.0.254)"
13:27:35.787	**ALERT:** 10.0.0.3 measured 186.5 Mbps by the switch (threshold 3.0)
13:27:35.790	**BLOCKED**, 3 ms later
13:27:50.814	**UNBLOCKED** after 15.0 s of silence

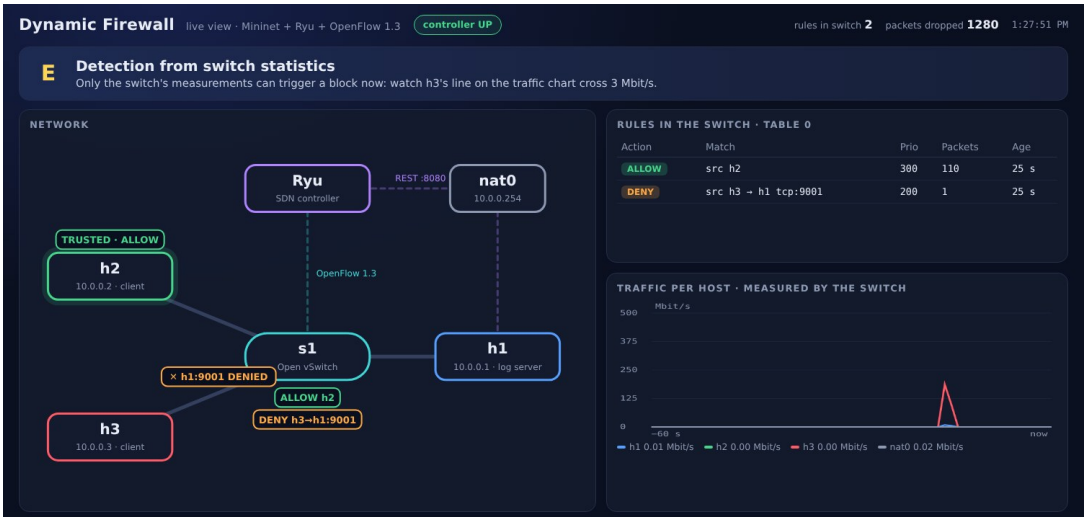
In the browser



Before: quiet network.



During: h3's line spikes on the traffic chart (about 190 Mbit/s, measured by the switch); h3 is red and a BLOCK row has been counting over a thousand dropped packets.



After: the BLOCK is gone; the spike remains on the chart as history. The DENY row is the one scenario D re-installed; see the known issue in chapter 10.

Result: detection by the switch works: h3 was measured at about 62 times the limit and blocked within the one-second sampling interval. The server stayed reachable throughout, which was **not** true in the original version (it blocked itself, a bug fixed here).

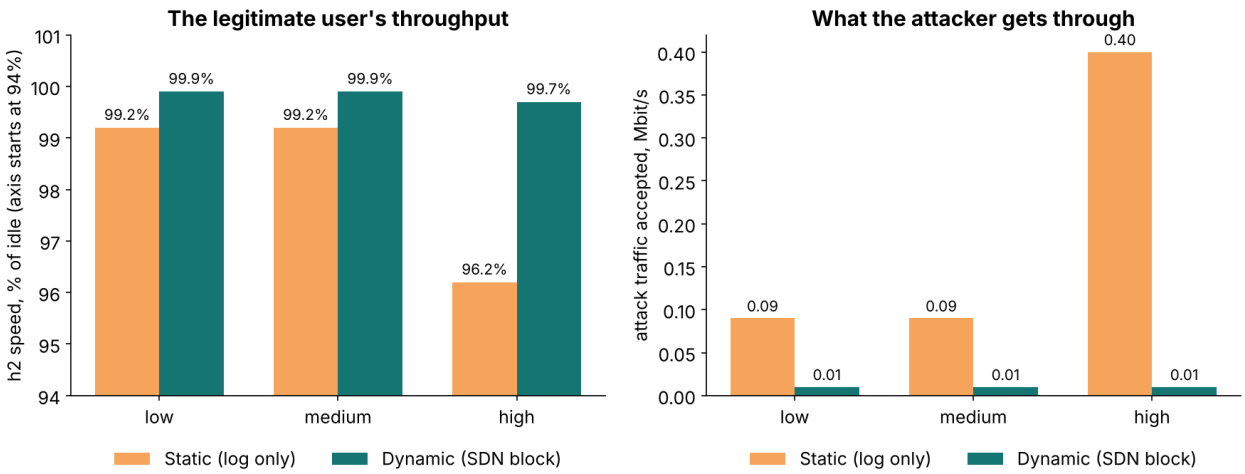
CHAPTER 9

How effective is it?

Measured on our VM: 3 runs of 20 seconds per condition, 10 Mbit/s server link

Method. h3 floods the log server while h2, the legitimate user, measures its own speed (iperf) and latency (ping). Three conditions: **idle** (no flood), **static** (the server detects but the network does not react) and **dynamic** (the SDN block is active). Three flood intensities: low (1 connection, 20 messages/s), medium (1 connection, unthrottled) and high (4 connections, unthrottled, enough to fill the server link).

1 · Does it stop the attack?



Left: h2's speed as a percentage of its idle speed (the axis starts at 94% to make small differences visible). Right: how much attack traffic the server still accepted.

Flood	Attack traffic: static	Attack traffic: dynamic	Reduction	Packets dropped by the switch
low	0.09 Mbit/s	0.01 Mbit/s	−89%	32 per trial
medium	0.09 Mbit/s	0.01 Mbit/s	−89%	34 per trial
high	0.40 Mbit/s	0.01 Mbit/s	−97%	157 per trial

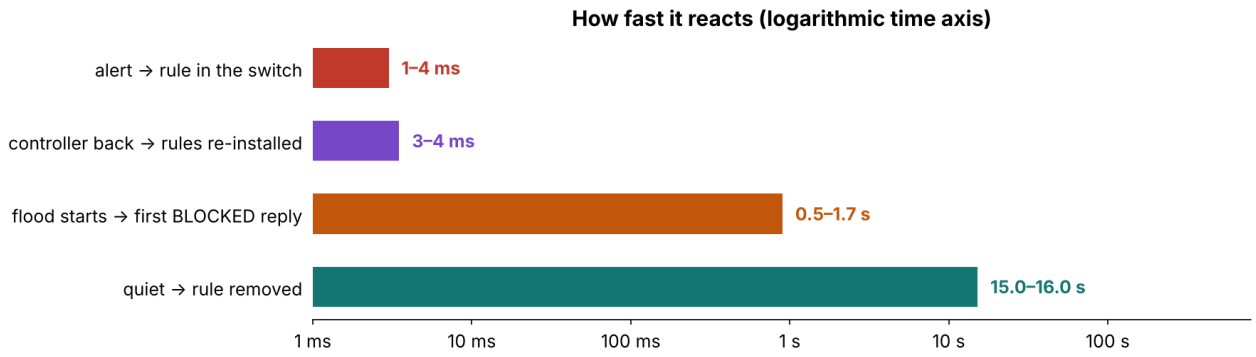
Yes. In every condition the attacker's traffic was cut by 89% to 97%. While blocked, h3 had no connectivity at all (100% packet loss on ping), and the switch's own counters show the dropped packets (the dashboard's counter reached 1,280 by the end of scenario E, almost all from E's block).

2 · Does it protect the legitimate user?

Flood	h2 speed: static	h2 speed: dynamic	h2 ping: static	h2 ping: dynamic
idle (reference)	9.57 Mbit/s		135.3 ms	
low	9.49 Mbit/s (99.2%)	9.56 Mbit/s (99.9%)	134.7 ms	134.7 ms
medium	9.49 Mbit/s (99.2%)	9.56 Mbit/s (99.9%)	135.7 ms	135.0 ms
high	**9.21 Mbit/s (96.2%)**	**9.54 Mbit/s (99.7%)**	**138.4 ms**	**135.4 ms**

Yes, but the gain is small in these tests. Under the heaviest flood h2 kept 99.7% of its speed with the firewall against 96.2% without, and its latency stayed at the idle value (+0.1 ms) instead of rising 3.1 ms. Under the low and medium floods the flood is too light to congest the link, so h2 loses under 1% either way: the firewall still stops the attacker, there is just little harm to prevent. (The ~135 ms ping includes h2's own speed test filling the link queue, so compare the differences, not the absolute values.)

3 · How fast does it react?



Reaction times, from the logs of the recorded runs and the experiments. Note the logarithmic axis.

Measurement	Value	Where it comes from
alert → rule in the switch	1–4 ms	server log, scenarios A, B and E
flood starts → first BLOCKED reply	0.5–1.7 s	experiments: the time for the 9th message to arrive
bandwidth detection	within 1 s	the statistics are sampled once a second
h3 silent → rule removed	15.0–16.0 s	recovery timer plus the 1-second sweep
controller back → rules re-installed	3–4 ms	scenario D log

The *decision* is fast (milliseconds); the time to the first BLOCKED reply is dominated by the policy itself, because the 9th message has to arrive. A lower threshold would block sooner, at the price of more false positives.

4 · Is it precise? (false positives and side effects)

- **The legitimate host was never blocked:** h2 is trusted, and in none of the scenarios or experiments did it receive a BLOCKED or DENIED reply.
- **Denies are surgical:** in scenario B, h3 lost one port and kept everything else.
- **No self-inflicted outage:** the server and controller are protected from bandwidth blocks (the original version blocked the server itself and locked the network up).
- **The trade-off:** a non-trusted host that legitimately sends more than 8 messages in 5 seconds *is* blocked for about 15 seconds. The threshold and window are one setting in `policy.json`.

5 · Is it resilient?

Scenario D showed it: with the controller down, all 20 flood messages were still answered, exactly one warning was logged, and the rules were back 3–4 ms after the controller returned. Existing rules stay in the switch while the controller is away (fail-static), so protection already in place is not lost.

6 · What does it cost?

Very little. The controller polls the switch once a second; a rule is one flow entry; each block or deny is one REST call that took under a millisecond on the controller (0.7 ms in Ryu's log). The experiments show no measurable penalty for the legitimate user in the idle and dynamic conditions beyond measurement noise (speeds within 0.01 Mbit/s across runs).

Summary

Scenario	Goal	Achieved?
A · flood	stop it quickly, then recover alone	Yes: 3 ms to block, 15.5 s to recover
B · wrong door	cut only the forbidden flow	Yes: h3 kept every other service
C · trusted host	never punish h2	Yes: 40 / 40 ACK, ALLOW counter 18 → 74
D · controller crash	keep serving and self-heal	Yes: one warning, rules back in 3–4 ms
E · switch measurement	detect by traffic, not messages	Yes: 186.5 Mbit/s blocked, server stayed up

CHAPTER 10

Limits and verdict

What it does not do, one known issue, and the bottom line

What it does not do

- **A small, emulated network.** One switch and three hosts, all inside Mininet on a single VM. Multiple switches, real links and real attack tools were not tested.
- **Light floods.** Only the high flood was strong enough to congest the 10 Mbit/s server link, so the measured benefit to the legitimate user is a few percent (Chapter 9).
- **Fixed thresholds.** 8 messages in 5 s and 3 Mbit/s are static numbers. A bursty but legitimate client is blocked for about 15 s, and an attacker who stays just under both limits is never blocked.
- **Trusted means fully exempt.** If h2 were compromised, its flood would pass straight through the ALLOW rule, as scenario C shows.
- **No new blocks while the controller is down.** Rules already in the switch keep working, but an attack that starts during the outage is not stopped until Ryu returns (scenario D).
- **Rate limiting covers port 9000 only.** Other ports are protected only by the allow-list (9001) or by the bandwidth rule (scenario E).

A known issue

When the controller came back in scenario D, the server re-installed h3's DENY rule (13:27:26). That rule was still in the switch during scenario E, with an age of 25 s at 13:27:51, about 43 s after the original DENY was installed at 13:27:08. So it outlived its 30-second lifetime. It is harmless here, because it only shuts port 9001 for h3, but a re-installed rule should keep the original expiry time. The fix is for the reconcile loop to re-install a rule only for the time it has left, and to remove it once that original expiry passes.

Conclusion

The project meets its objectives. Using a Python TCP server, the Ryu controller and Open vSwitch, we built a firewall whose rules change while the network runs. A misbehaving host is detected by the application or by the switch's own counters, dropped at the switch a few milliseconds after the decision, and released automatically once it goes quiet. The response is proportionate: BLOCK for a flooding host, DENY for one forbidden flow. The trusted host was never blocked, and after a controller crash the system kept serving and restored its rules by itself. The experiments show attack traffic reaching the server cut by 89% to 97% at very little cost; the benefit to the legitimate user was small only because the test floods were light.

Future work: adaptive thresholds instead of fixed ones, larger topologies with heavier floods, and the rule-lifetime fix above.